



Αυτόνομη Πλοήγηση Drone με Αποφυγή Εμποδίων μέσω Αλγορίθμων Αναζήτησης Διαδρομής

- Σπυρίδων Παπαδόπουλος [7574] ~ Σταύρος Μανιάκας [7687] ~ Ροδόπουλος Κωνσταντίνος [7708] .

1. Περιγραφή του Συγκεκριμένου Προβλήματος

Η αυτόνομη πλοήγηση μη επανδρωμένων εναέριων οχημάτων (UAVs) αποτελεί ένα από τα πιο ενδιαφέροντα και σύγχρονα προβλήματα στη ρομποτική και την εφαρμοσμένη μαθηματική ανάλυση. Τα αυτόνομα drones βρίσκουν πλέον εφαρμογή σε πληθώρα πεδίων, όπως η επιτήρηση περιοχών, η χαρτογράφηση και οι αποστολές έρευνας και διάσωσης.

Σε όλες αυτές τις εφαρμογές, το drone πρέπει να είναι ικανό να υπολογίζει με ακρίβεια μία ασφαλή και αποδοτική διαδρομή, εκκινώντας από μια αρχική θέση και καταλήγοντας σε έναν τελικό στόχο, αποφεύγοντας παράλληλα φυσικά εμπόδια στον χώρο. Το πρόβλημα αυτό διατυπώνεται ως ένα κατεχοχόν πρόβλημα βελτιστοποίησης, όπου αναζητείται η ελάχιστη τιμή μιας κατάλληλα ορισμένης συνάρτησης κόστους.

Σκοπός της παρούσας εργασίας είναι η ανάπτυξη και σύγκριση πολλαπλών αλγορίθμων (**A**, **Dijkstra**, **Grassfire**) για την αυτόνομη πλοήγηση ενός drone σε 2D περιβάλλον με εμπόδια, όπου η τελική τροχιά θα αποτελείται από διακριτά σημεία. Η λύση προγραμματίστηκε σε γλώσσα Python με στόχο να εφαρμοστεί σε πραγματικά εκπαιδευτικά drones του εργαστηρίου, τα οποία ελέγχονται μέσω κατάλληλων προγραμματιστικών εντολών.

2. Μαθηματική Μοντελοποίηση.

Για την επίλυση του προβλήματος της αυτόνομης πλοήγησης, η γεωμετρία του πραγματικού χώρου και οι κινηματικοί περιορισμοί του drone πρέπει να μεταφραστούν σε ένα αυστηρό μαθηματικό μοντέλο. Στην παρούσα εργασία, η μοντελοποίηση αυτή αντιστοιχίζεται άμεσα με τις δομές δεδομένων και τις συναρτήσεις που αναπτύχθηκαν στο [αρχείο gui.py](#).

A. Διακριτοποίηση και Αναπαράσταση του Χώρου Πτήσης

Ο φυσικός χώρος πτήσης του drone προσεγγίζεται ως ένα δισδιάστατο ορθογώνιο πλέγμα (2D Grid) με διακριτές τιμές. Το μέγεθος του πλέγματος καθορίζεται δυναμικά από τον χρήστη μέσω των μεταβλητών M (γραμμές – row_input) και N (στήλες – cols_input).

Κάθε σημείο (κόμβος) στον χώρο αναπαρίσταται ως ένα διακριτό ζεύγος συντεταγμένων της μορφής: $n = (r, c)$ όπου r ο δείκτης της γραμμής και c ο δείκτης της στήλης με τους εξής περιορισμούς: $0 \leq r \leq M, 0 \leq c \leq N$

Στο μοντέλο αυτό ορίζονται τρία κύρια υποσύνολα καταστάσεων:

1. **Κόμβος Αφετηρίας (Start Node):** Ένα μοναδικό σημείο $S(r_s, c_s) \in R^2$ που ορίζει τη θέση απογείωσης.
2. **Κόμβος Στόχου (Goal Node):** Ένα μοναδικό σημείο $G(r_g, c_g) \in R^2$ που ορίζει το επιθυμητό σημείο τερματισμού της αποστολής.
3. **Σύνολο Εμποδίων (Obstacles Set - O):** Ένα δυναμικό σύνολο (Python set) που περιέχει όλες τις συντεταγμένες των κελιών που είναι κατειλημμένα από φυσικά εμπόδια και στα οποία απαγορεύεται η μετάβαση του οχήματος:

$$O = \{(r_1, c_1), (r_2, c_2), \dots, (r_k, c_k)\}$$

B. Κινηματικοί Περιορισμοί και Μετάβαση Καταστάσεω

Το drone μοντελοποιείται ως ένα σημειακό αντικείμενο στον κάναβο. Οι κινηματικοί περιορισμοί κίνησής ελέγχονται από τον πίνακα μετατοπίσεων κατά τη διάρκεια εξέτασης των γειτονικών κόμβων. Το όχημα επιτρέπεται να κινηθεί αποκλειστικά κατά μήκος των κύριων αξόνων.

Η μαθηματική μετάβαση από έναν τρέχοντα κόμβο (r, c) σε έναν γειτονικό κόμβο (r', c') ορίζεται από το σύνολο επιτρεπτών μετατοπίσεων:

$$\Delta = \{(0, 1), (1, 0), (0, -1), (-1, 0)\}$$

$$(r', c') = (r + \Delta r, c + \Delta c), \text{ όπου } (\Delta r, \Delta c) \in \Delta$$

Για να είναι έγκυρη η μετάβαση, πρέπει να ικανοποιείται η παρακάτω λογική συνθήκη:

$$(0 \leq r' < M) \wedge (0 \leq c' < N) \wedge ((r', c') \notin O)$$

C. Μαθηματική Διατύπωση της Συνάρτησης Κόστους

Ο αλγόριθμος A* αναζητά το μονοπάτι που ελαχιστοποιεί τη συνολική συνάρτηση κόστους : $f(n) = g(n) + h(n)$

- **Πραγματικό Κόστος $g(n)$** : Αντιπροσωπεύει τη μαθηματική απόσταση από την αφετηρία S μέχρι τον τρέχοντα κόμβο n . Δεδομένου ότι η κίνηση γίνεται από κελί σε κελί, το κόστος μετάβασης μεταξύ δύο έγκυρων γειτονικών κόμβων είναι σταθερό και ίσο με τη μονάδα ($c(n, n') = 1$). Συνεπώς, στον κώδικα το κόστος υπολογίζεται αναδρομικά ως: $g(n') = g(n) + 1$
- **Ευρετικό Κόστος $h(n)$** : Επειδή το drone δεν εκτελεί διαγώνιες κινήσεις, η Ευκλείδεια απόσταση θα υποτιμούσε το πραγματικό κόστος κίνησης. Για τον λόγο αυτό, χρησιμοποιείται η **Απόσταση Manhattan (Manhattan Distance)**, η οποία ορίζει τη θεωρητική απόσταση μεταξύ του τρέχοντος κόμβου $n(r_n, c_n)$ και του στόχου $G(r_G, c_G)$, ως το άθροισμα των απόλυτων διαφορών των συντεταγμένων τους: $h(n) = |r_n - r_G| + |c_n - c_G|$

D. Μοντελοποίηση Του Μητρώου Καταστάσεων

Μόλις ο αλγόριθμος *astar* επιστρέψει τη βέλτιστη ακολουθία κόμβων (τροχιά), η συνάρτηση *create_path_matrix* αναλαμβάνει τη χαρτογράφηση αυτής της πληροφορίας σε ένα δισδιάστατο μητρώο καταστάσεων $A_{M \times N}$. Κάθε στοιχείο a_{ij} του μητρώου λαμβάνει μια ακέραια τιμή που υποδηλώνει τον λειτουργικό χαρακτήρα του συγκεκριμένου σημείου στον χώρο.

Τιμές a_{ij} :

- 0 , ελεύθερος χώρος / μη επισκέψιμος κόμβος
- 1 , κόμβος αφετηρίας
- 2 , κόμβος στόχου
- 3 , κόμβος που ανήκει στη βέλτιστη τροχιά

Αυτό το ψηφιακό μητρώο A αποτελεί τη μαθηματική είσοδο για τη συνάρτηση παραγωγής εντολών (*commandgen*), η οποία μεταφράζει τις μεταβολές των δεικτών i, j των στοιχείων με τιμή 3 και 2 σε διαδοχικά διανύσματα κίνησης του drone (π.χ. $v_{forward}, v_{right}$). Το μέτρο αυτών των διανυσμάτων καθορίζεται δυναμικά από τη μεταβλητή *step* η οποία έχει ως προεπιλεγμένη τιμή τα **30cm**. Η τιμή αυτή επιλέχθηκε έπειτα από εργαστηριακές δοκιμές ως η χρυσή τομή για τη σταθερότητα της πτήσης, καθώς επιτρέπει στο σύστημα οπτικού εντοπισμού θέσης του Tello να αναγνωρίζει με ακρίβεια τη μετατόπιση στο έδαφος, εξαλείφοντας το φαινόμενο της ολίσθησης που εμφανίζεται σε μικρότερα βήματα.

3. Περιγραφή των Αλγορίθμων

Η υλοποίηση του συστήματος βασίζεται στη συνεργασία δύο διακριτών αλγοριθμικών διαδικασιών: τον αλγόριθμο αναζήτησης και υπολογισμού της διαδρομής (*astar*) και τον αλγόριθμο σειριακής παραγωγής εντολών (*commandgen*). Παρακάτω αναλύεται η προγραμματιστική τους λογική και οι δομές δεδομένων που επιλέχθηκαν.

A. Αλγόριθμος Αναζήτησης A*

Ο αλγόριθμος A* υλοποιήθηκε με στόχο τη βέλτιστη διαχείριση της μνήμης και την ταχεία εκτέλεση της αναζήτησης. Για τον σκοπό αυτό, αποφεύχθηκε η χρήση απλών λιστών για την αναζήτηση κόμβων και ενσωματώθηκαν εξειδικευμένες δομές δεδομένων.

Δομές Δεδομένων :

- **Ουρά Προτεραιότητας (Min-Heap):** Η λίστα ανοιχτών κόμβων (*front*) διαχειρίζεται μέσω της βιβλιοθήκης *heapq* της Python. Η δομή αυτή εγγυάται ότι η εισαγωγή νέων κόμβων και η εξαγωγή του κόμβου με το μικρότερο συνολικό κόστος εκτελούνται σε λογαριθμικό χρόνο.
- **Πίνακες Καταγραφής (Hash Maps):** Χρησιμοποιήθηκαν δύο λεξικά (dictionaries), το *cost* και το *came_from*, για την αποθήκευση του κόστους κάθε κόμβου και του γονικού του κόμβου αντίστοιχα. Οι δομές αυτές προσφέρουν χρόνο πολυπλοκότητας $O(1)$ για την αναζήτηση και ανάκτηση τιμών.

Αλγοριθμική Ροή :

1. **Αρχικοποίηση:** Ο αλγόριθμος εισάγει τον κόμβο εκκίνησης στην ουρά προτεραιότητας και αρχικοποιεί τα λεξικά.
2. **Βρόχος Εξέτασης (While Loop):** Όσο υπάρχουν κόμβοι προς εξέταση στην ουρά, ο αλγόριθμος εξάγει (*heappop*) τον κόμβο με την υψηλότερη προτεραιότητα. Αν αυτός είναι ο στόχος, ο βρόχος διακόπτεται.
3. **Έλεγχος Γειτονικών Κόμβων:** Ο κώδικας διατρέχει μια λίστα μετατοπίσεων και ελέγχει τους 4 γείτονες του τρέχοντος κόμβου. Με απλές λογικές συνθήκες (*if*), απορρίπτει άμεσα όσους γείτονες βρίσκονται εκτός των ορίων του πίνακα ή ανήκουν στη δομή συνόλου των εμποδίων (*obstacles*).
4. **Ενημέρωση και Εισαγωγή:** Αν ο γείτονας είναι προσπελάσιμος και επιτυγχάνει μικρότερο κόστος από προηγούμενες προσπάθειες, ενημερώνονται τα λεξικά και εισάγεται (*heappush*) ξανά στην ουρά.

5. **Ανακατασκευή Μονοπατιού (Backtracking):** Στο τέλος της εκτέλεσης, χρησιμοποιείται ένας βρόχος **while** που διατρέχει το λεξικό **came_from** από τον στόχο προς την αφετηρία, δημιουργώντας μια λίστα συντεταγμένων, η οποία στη συνέχεια αναστρέφεται (**path.reverse()**).

B. Αλγόριθμος Αναζήτησης Dijkstra.

Ο αλγόριθμος Dijkstra αναπτύχθηκε ως μια εναλλακτική μέθοδος εύρεσης της συντομότερης διαδρομής. Αντίθετα με τον A*, δεν χρησιμοποιεί συνάρτηση εκτίμησης (heuristic), αλλά εξερευνά τον χώρο υπολογίζοντας αποκλειστικά το πραγματικό συσσωρευμένο κόστος από την αφετηρία. Η υλοποίησή του (συνάρτηση **dijkstra**) αξιοποιεί επίσης Ουρά Προτεραιότητας (**heapq**) για βέλτιστη διαχείριση μνήμης και χρόνου. Παρότι εγγυάται την εύρεση του βέλτιστου μονοπατιού, η απουσία ευρετικής κατεύθυνσης τον καθιστά πιο «απαιτητικό» υπολογιστικά σε ανοιχτούς χώρους, καθώς επεκτείνεται ακτινωτά προς όλες τις κατευθύνσεις.

C. Αλγόριθμος Αναζήτησης Grassfire.

Ο αλγόριθμος Grassfire αποτελεί μια διαφορετική προσέγγιση, βασισμένη στην Αναζήτηση κατά Πλάτος. Εκκινεί από τον κόμβο Στόχο και "εξαπλώνεται" διαδοχικά σε όλα τα ελεύθερα γειτονικά κελιά, λειτουργώντας μέσω μιας απλής δομής Ουράς (**deque**). Κάθε κελί λαμβάνει μια ακέραια τιμή απόστασης από τον στόχο. Μόλις το «κύμα» φτάσει στην αφετηρία, ο αλγόριθμος ανακατασκευάζει το μονοπάτι ακολουθώντας αυστηρά φθίνουσα ακολουθία τιμών προς τα κάτω μέχρι να επιστρέψει στον στόχο. Είναι εξαιρετικά αξιόπιστος σε κλειστούς λαβυρίνθους, αλλά απαιτεί την πλήρη χαρτογράφηση των αποστάσεων πριν την παραγωγή του μονοπατιού.

D. Αλγόριθμος Παραγωγής Εντολών.

Ο δεύτερος αλγόριθμος αναλαμβάνει να μετατρέψει τον πίνακα καταστάσεων σε ένα εκτελέσιμο αρχείο κειμένου (**command.txt**). Η λειτουργία του είναι αυστηρά σειριακή και βασίζεται σε ελέγχους γειννίας (**lookahead checks**).

Αλγοριθμική Ροή :

1. **Εντοπισμός Αφετηρίας:** Ο αλγόριθμος σαρώνει αρχικά όλον τον πίνακα μέσω διπλού βρόχου **for** για να εντοπίσει τις συντεταγμένες εκκίνησης (κελί με τιμή 1) και αποθηκεύει τους δείκτες θέσης **cx** και **cy**.
2. **Αρχικοποίηση Αρχείου:** Ανοίγει το αρχείο εξόδου και εγγράφει τις βασικές εντολές ενεργοποίησης και απογείωσης (π.χ., **streamon**, **takeoff**, **up**).

3. **Βρόχος Πλοήγησης (While Loop):** Ο κύριος βρόχος εκτελείται συνεχώς με τη χρήση της δομής **while True**, διασφαλίζοντας τη σειριακή επεξεργασία του μονοπατιού. Σε κάθε επανάληψη, το τρέχον κελί μηδενίζεται (**matrix[cy][cx] = 0**) για την αποφυγή ατέρμονων βρόχων ή αναδρομών. Στη συνέχεια, ο κώδικας ελέγχει μέσω διαδοχικών εντολών **if/elif** τα γειτονικά κελιά. Μόλις εντοπιστεί η κατεύθυνση του μονοπατιού (τιμή 3) ή ο τελικός στόχος (τιμή 2), ο αλγόριθμος εγγράφει την αντίστοιχη κατευθυντική εντολή στο αρχείο **command.txt** (π.χ., **forward, left, right, back**), πλαισιωμένη από την τρέχουσα τιμή της παγκόσμιας μεταβλητής **step**. Αμέσως μετά, ανανεώνονται οι δείκτες θέσης **cx** ή **cy** για την επόμενη επανάληψη.
4. **Τερματισμός:** Μόλις οι δείκτες φτάσουν στον στόχο, ο βρόχος σπάει και ο αλγόριθμος εγγράφει τις τελικές εντολές επίδειξης και προσγείωσης (**flip f, land**).

4. Κώδικας Υλοποίησης και Αρχιτεκτονική Συστήματος

Η ανάπτυξη του λογισμικού πραγματοποιήθηκε εξ ολοκλήρου στη γλώσσα προγραμματισμού Python. Η αρχιτεκτονική του συστήματος είναι σπονδυλωτή διαχωρίζοντας το γραφικό περιβάλλον (Frontend) από τη δικτυακή επικοινωνία με το υλικό (Backend). Το σύστημα απαρτίζεται από τρία κύρια υποσυστήματα.

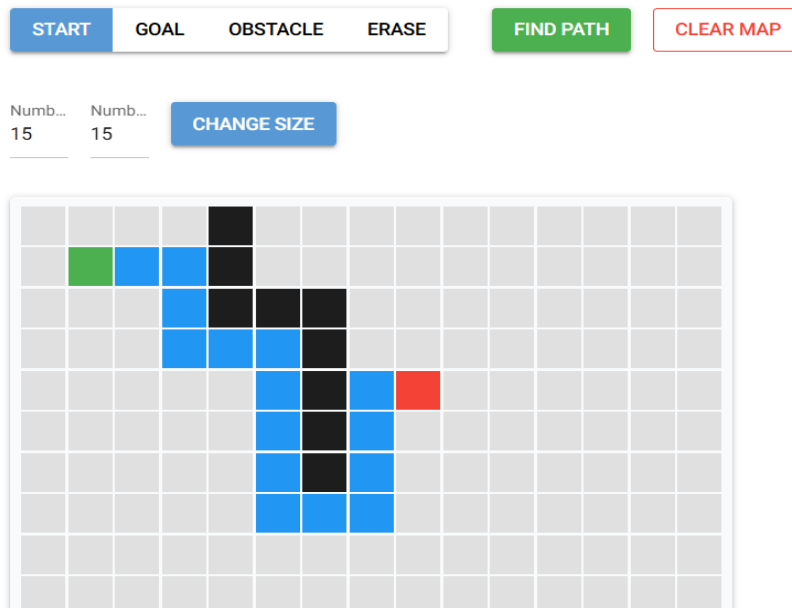
A. Γραφική Διεπαφή και Διαχείριση Συστήματος (*gui.py*)

Για την υλοποίηση του γραφικού περιβάλλοντος (GUI) επιλέχθηκε το σύγχρονο πλαίσιο **NiceGUI**, το οποίο βασίζεται στο **FastAPI**. Το πλαίσιο αυτό επιτρέπει τη δημιουργία διαδραστικών Web εφαρμογών, αξιοποιώντας τις ασύγχρονες δυνατότητες (**async/await**) της Python.

- **Διαχείριση Χρηστών και Ασφάλεια:** Η εφαρμογή δεν είναι ανοιχτή στο κοινό. Υλοποιήθηκε ένα ολοκληρωμένο σύστημα αυθεντικοποίησης (Login System) που αντλεί δεδομένα από τοπικά αρχεία ρυθμίσεων (**users.json, admins.json**). Μέσω εξειδικευμένων φίλτρων ελέγχου, όπως το **AuthMiddleware** και το **AdminMiddleware**, το σύστημα αναχαιτίζει τα αιτήματα HTTP και ανακατευθύνει αυτόματα τους μη συνδεδεμένους χρήστες στη σελίδα εισόδου.
- **Διαδραστικά Grid Αναζήτησης (Σελίδες */astar, /dijkstra, /grassfire*):** Για την εξυπηρέτηση των τριών διαφορετικών αλγορίθμων δημιουργήθηκαν τρεις ξεχωριστές σελίδες στο γραφικό περιβάλλον. Εδώ φιλοξενείται η οπτικοποίηση

της αναζήτησης. Ο χρήστης μπορεί να ορίσει δυναμικά το μέγεθος του πλέγματος και να τοποθετήσει την αφετηρία, τον στόχο και τα εμπόδια (δημιουργώντας τοπικά το Path Matrix). Με την εκτέλεση της αναζήτησης, το προκύπτον μονοπάτι αποτυπώνεται οπτικά και καλείται στο παρασκήνιο η συνάρτηση **commandgen** για την παραγωγή του εκτελέσιμου αρχείου **command.txt**.

A* Pathfinding Interface



- **Σελίδα Δυναμικής Ρύθμισης Βήματος (Σελίδα /step)** Για την αποφυγή της στατικής εισαγωγής παραμέτρων στον κώδικα, αναπτύχθηκε μια ανεξάρτητη σελίδα διαχείρισης του βήματος πτήσης. Η σελίδα αυτή ενσωματώνει το στοιχείο εισαγωγής αριθμών **ui.number** του **NiceGUI**. Για λόγους ασφαλείας και συμβατότητας με το υλικό, έχουν οριστεί αυστηρά όρια εγκυρότητας τιμών, με **ελάχιστο τα 20cm** και **μέγιστο τα 500cm**, ακολουθώντας πιστά τους περιορισμούς που επιβάλλει το επίσημο Tello SDK. Όταν ο χρήστης εισάγει μια νέα τιμή και πιέζει το κουμπί "**Update**", ενεργοποιείται η συνάρτηση **update_step()**, η οποία αντλεί με ασφάλεια την τιμή από το γραφικό στοιχείο, τη μετατρέπει σε ακέραιο αριθμό (**int**) και ενημερώνει την παγκόσμια μεταβλητή **step** στο Backend, αλλάζοντας άμεσα τη συμπεριφορά των επόμενων αποστολών.

B. Επικοινωνία με το Drone (tello.py)

Η επικοινωνία χαμηλού επιπέδου με τον μικροελεγκτή του DJI Tello drone υλοποιείται μέσω της κλάσης **Tello**.

- **Πρωτόκολλο UDP:** Το drone λειτουργεί ως Access Point (AP). Ο υπολογιστής συνδέεται στο δίκτυο του drone και ανοίγει ένα UDP Socket στη θύρα **8889**. Μέσω αυτού του socket, το πρόγραμμα αποστέλλει τις εντολές κίνησης (π.χ. *forward 50*) σε μορφή κωδικοποίησης UTF-8.
- **Ασύγχρονη Λήψη Επιβεβαιώσεων (Multithreading):** Το Tello SDK απαιτεί την παραλαβή μιας επιβεβαίωσης (**OK** ή **Error**) μετά από κάθε εντολή πριν δεχθεί την επόμενη. Για να μην "παγώσει" η διεπαφή χρήστη κατά την αναμονή αυτής της απάντησης, υλοποιήθηκε ένα ανεξάρτητο νήμα (background thread) ονόματι **_receive_thread**. Αυτό το νήμα εκτελείται παράλληλα, "ακούγοντας" διαρκώς τη θύρα του socket και καταγράφοντας τις απαντήσεις.

C. Διαχείριση Ροής Βίντεο (Video Streaming).

Η επιτήρηση του χώρου σε πραγματικό χρόνο είναι κρίσιμη για την αποστολή του drone. Η διαδικασία αυτή ελέγχεται από την κλάση **VideoStreamer**.

- **Αποκωδικοποίηση με OpenCV:** Το Tello εκπέμπει τη ροή βίντεο μέσω H.264 στην UDP θύρα **11111**. Χρησιμοποιώντας τη βιβλιοθήκη Μηχανικής Όρασης **OpenCV** (*cv2.VideoCapture*), το πρόγραμμα δεσμεύει αυτά τα καρέ.
- **Βελτιστοποίηση και Απεικόνιση:** Για τη μείωση του φόρτου δικτύου και την ομαλή αναπαραγωγή στον Web Browser, κάθε καρέ υφίσταται αλλαγή μεγέθους (resize σε ανάλυση 640x480) και συμπιέζεται σε μορφή JPEG. Η ροή αποστέλλεται έπειτα ασύγχρονα στο Frontend (στη σελίδα **/dashboard**) μέσω του **FastAPI** ως **StreamingResponse** τύπου **multipart/x-mixed-replace**.

D. Έλεγχος Πτήσης & Τηλεμετρία (Σελίδα **/dashboard**)

Στη σελίδα του Dashboard, η έναρξη της πτήσης δεν είναι αυτοματοποιημένη προκειμένου να διασφαλιστεί η ασφάλεια στον χώρο του εργαστηρίου. Ο χρήστης έχει τον πλήρη έλεγχο εκκίνησης μέσω του ειδικού κουμπιού **"START MISSION"**, το οποίο καλεί την ασύγχρονη συνάρτηση **auto_start**. Επιπλέον, με το πέρας της αποστολής και την προσγείωση του drone, το λογισμικό επικοινωνεί δυναμικά με το υλικό μέσω της εντολής **get_battery()**. Το τελικό ποσοστό της μπαταρίας εξάγεται επιτυχώς και προβάλλεται στον χρήστη ως οπτική ειδοποίηση (**UI notification**), παρέχοντας κρίσιμα δεδομένα τηλεμετρίας.

5. Πειραματικά Αποτελέσματα

Η αξιολόγηση του συστήματος αυτόνομης πλοήγησης πραγματοποιήθηκε σε πραγματικές συνθήκες πτήσης με τη χρήση του εκπαιδευτικού drone DJI Tello. Σκοπός των πειραμάτων ήταν η επαλήθευση της ορθής εκτέλεσης των αρχείων εντολών (**command.txt**) που παρήγαγαν οι αλγόριθμοι αναζήτησης, καθώς και η μελέτη της δυναμικής συμπεριφοράς του οχήματος.

Οι πτητικές δοκιμές έλαβαν χώρα σε δύο διαφορετικά περιβάλλοντα:

- Σε **κλειστό χώρο** εργαστηρίου (απουσία εξωτερικών διαταραχών).
- Σε **ανοιχτό εξωτερικό χώρο**, προκειμένου να αξιολογηθεί η σταθερότητα του drone υπό την επίδραση αεροδυναμικών διαταραχών (άνεμος).

A. Επίδραση Ενεργειακής Στάθμης και Πτώση Τάσης

Κατά τη διάρκεια των επαναλαμβανόμενων δοκιμών, παρατηρήθηκε μια **άμεση συσχέτιση μεταξύ της στάθμης της μπαταρίας και της κινηματικής ακρίβειας του drone**. Ειδικότερα, όταν το ποσοστό της μπαταρίας (όπως αυτό ανακτάται δυναμικά από τη διεπαφή τηλεμετρίας του λογισμικού) υποχωρούσε κάτω από το **30%**, καταγράφηκαν οι εξής σημαντικές αλλοιώσεις στη συμπεριφορά του υλικού:

- **Απώλεια Χωρικής Ακρίβειας (Drift):** Λόγω της πτώσης τάσης στους ακροδέκτες της μπαταρίας, οι ηλεκτροκινητήρες αδυνατούσαν να παράξουν τη βέλτιστη ώθηση με σταθερό ρυθμό. Ως αποτέλεσμα, η εκτέλεση του προκαθορισμένου βήματος (π.χ. **step = 30**) έχανε την ακρίβειά της, οδηγώντας σε ολίσθηση (**drift**) του οχήματος και απόκλιση από την ιδεατή, θεωρητική τροχιά που είχε υπολογίσει ο αλγόριθμος στο πλέγμα.
- **Ενεργοποίηση Μηχανισμών Ασφαλείας (Ακύρωση Ακροβατικών):** Σε συνθήκες χαμηλής τάσης (<30%), διαπιστώθηκε ότι ο εσωτερικός ελεγκτής του Tello SDK αγνοεί στοχευμένα την εκτέλεση ενεργοβόρων εντολών. Συγκεκριμένα, η τελική εντολή επίδειξης (**flip f**), η οποία προγραμματίζεται στο τέλος κάθε αποστολής, δεν εκτελούνταν. Η παράλειψη αυτή αποτελεί ενσωματωμένο μηχανισμό προστασίας του κατασκευαστή, προκειμένου να αποτραπεί η απότομη αποφόρτιση και η ενδεχόμενη πτώση του οχήματος.

B. Επίδραση του Μεγέθους Βήματος στην Ακρίβεια Κίνησης

Κατά τη φάση παραμετροποίησης του συστήματος, αξιολογήθηκε η πτητική συμπεριφορά του drone για διάφορες τιμές της μεταβλητής του διανύσματος κίνησης (**step**). Από τις **δοκιμές προέκυψε ότι η χρήση πολύ μικρών βημάτων, της τάξεως των 10 έως 20 εκατοστών, υποβαθμίζει σημαντικά την ακρίβεια της αυτόνομης πλοήγησης.**

Λόγω της αδράνειας του οχήματος κατά την εκκίνηση και την πέδηση των κινητήρων, καθώς και των κατασκευαστικών ανοχών του οπτικού συστήματος εντοπισμού (Vision Positioning System) του Tello, οι εντολές τόσο μικρής μετατόπισης δεν παράγουν μια διακριτή και ξεκάθαρη κίνηση στον χώρο. Αντιθέτως, το drone προσπαθεί να διορθώσει τη θέση του διαρκώς, με αποτέλεσμα η συνολική τροχιά να εμφανίζεται **ασταθής και η κίνηση να μη διαχωρίζεται σε διακριτά "κελιά" του Grid**. Το πειραματικό αυτό εύρημα επιβεβαιώνει και αιτιολογεί την τελική επιλογή των **30cm** ως την ελάχιστη βέλτιστη τιμή για την ασφαλή και ευκρινή λειτουργία του προτεινόμενου συστήματος.

6. Συμπεράσματα

Στην παρούσα εργασία αναπτύχθηκε και αξιολογήθηκε επιτυχώς ένα ολοκληρωμένο σύστημα αυτόνομης πλοήγησης για μη επανδρωμένα εναέρια οχήματα (UAVs) σε περιβάλλον με εμπόδια. Η μαθηματική μοντελοποίηση του προβλήματος μέσω δισδιάστατου Grid και η υλοποίηση τριών διαφορετικών αλγορίθμων εύρεσης διαδρομής (A^* , Dijkstra, Grassfire) παρείχαν ένα ισχυρό θεωρητικό υπόβαθρο για τη μελέτη και σύγκριση της βέλτιστης χωρικής αναζήτησης.

Από την υλοποίηση του λογισμικού προέκυψε ένα σύγχρονο, φιλικό και ασφαλές γραφικό περιβάλλον (**GUI**), το οποίο γεφυρώνει τη θεωρητική αλγοριθμική ανάλυση με τον πρακτικό έλεγχο υλικού (**τηλεμετρία, video streaming, εκτέλεση πτήσης**).

Τα πειραματικά αποτελέσματα στον φυσικό χώρο επιβεβαίωσαν την απόλυτη ορθότητα παραγωγής εντολών από τους αλγορίθμους. Παράλληλα, ανέδειξαν τους κρίσιμους φυσικούς και κατασκευαστικούς περιορισμούς του υλικού. Διαπιστώθηκε ότι η ενεργειακή στάθμη (πτώση τάσης της μπαταρίας κάτω από 30%) και η επιλογή πολύ μικρού βήματος μετατόπισης (κάτω από 20cm) αποτελούν τους σημαντικότερους παράγοντες υποβάθμισης της πτητικής ακρίβειας, προκαλώντας χωρική ολίσθηση (drift).

7. Μελλοντικές Επεκτάσεις

Μια μελλοντική εξέλιξη του συστήματος θα μπορούσε να ενσωματώσει αλγορίθμους Μηχανικής Όρασης (**Computer Vision**), αξιοποιώντας την ήδη υπάρχουσα ροή βίντεο (**H.264 stream**). Με αυτόν τον τρόπο, το σύστημα θα μπορούσε να αναγνωρίζει δυναμικά και απρόβλεπτα εμπόδια στον χώρο σε πραγματικό χρόνο, αναπροσαρμόζοντας τα βάρη του γραφήματος αναζήτησης εν πτήση, χωρίς να απαιτείται η εκ των προτέρων, χειροκίνητη χαρτογράφησή τους.